
✓ SEZIONE 1: VERO/FALSO (18 domande ricorrenti)

● BLOCCO A: Accessibilità (pattern più frequenti)

Q1. Un test esaustivo dell'accessibilità di una pagina web può essere fatto in modo automatico

- ☐ VERO
☒ FALSO

Motivazione: I test automatici sono rapidi ed economici ma non individuano tutti i problemi di accessibilità evidenti a un controllo umano. Esempio critico: strumenti come Accessiway inseriscono automaticamente attributi alt nelle immagini ma spesso in modo errato o non significativo. I test manuali sono quindi la maggioranza dei controlli necessari per una valutazione esaustiva dell'accessibilità.

Varianti possibili:

- "I test automatici sono sufficienti per verificare l'accessibilità" → FALSO
- "I test manuali sono più numerosi dei test automatici per l'accessibilità" → VERO

Q2. La validazione del codice XHTML e CSS è condizione sufficiente per garantire l'accessibilità

- ☐ VERO
☒ FALSO

Motivazione: È condizione **necessaria ma NON sufficiente**. Per garantire accessibilità completa servono: contrasto colori adeguato (WCAG 2.1), testi alternativi significativi, navigabilità da tastiera, uso corretto screen reader, struttura semantica, ordine di lettura logico, gestione del focus, ecc.

Q3. Permettere la fruibilità tramite screen reader è condizione necessaria e sufficiente per definire una pagina accessibile

- ☐ VERO
☒ FALSO

Motivazione: È necessaria ma non sufficiente. Accessibilità include anche: contrasto colori (utenti ipovedenti), navigazione da tastiera (disabilità motorie), comprensibilità contenuti (disabilità cognitive), compatibilità con tecnologie assistive diverse, dimensioni testi modificabili, evitare epilessia fotosensibile.

Q4. L'uso di menù a scomparsa non influisce sul grado di accessibilità

- ☐ VERO
☒ **FALSO**

Motivazione: I menù dropdown creano problemi: (1) se mal implementati, non tutte le voci sono raggiungibili via TAB; (2) menù non evidenziato fa perdere informazioni cruciali; (3) problemi con navigazione tastiera e gestione focus; (4) difficoltà su touch screen; (5) utenti con tremori possono perdere il menù.

Q5. L'utilizzo dei colori non è soggetto alle regole di accessibilità

- ☐ VERO
☒ **FALSO**

Motivazione: L'informazione non deve MAI essere veicolata unicamente tramite colore (WCAG 2.1 - Criterio 1.4.1). Necessario: contrasto sufficiente testo/sfondo (rapporto minimo 4.5:1 per testo normale, 3:1 per testo grande), considerare daltonismo (8% maschi, 0.5% femmine), alternative testuali, pattern/icone aggiuntive.

Q6. Le WCAG richiedono di identificare in modo diverso i link visitati dai non visitati

- ☒ **VERO**
☐ FALSO

Motivazione: WCAG 2.1 Criterio 1.4.1 richiede che l'informazione non sia veicolata solo tramite colore. Link visitati/non visitati devono essere distinguibili anche senza colore: sottolineatura diversa, peso font differente, icone, pattern. Aiuta orientamento utente e riduce carico cognitivo.

Variante correlata: "Per distinguere link visitati/non visitati basta cambiarne il colore" → FALSO

● BLOCCO B: Struttura e Web Design

Q7. Una struttura gerarchica deve essere ampia e poco profonda

☒ ~~VERO~~

☐ FALSO

Motivazione: Profondità ottimale: **4-5 livelli** (max 7 se informazione molto specifica). La profondità corrisponde ai clic dalla home alla pagina target. Troppo profonda = troppi clic, frustrazione, abbandono. Troppo ampia = sovraccarico cognitivo (max 7-10 voci per livello). Per siti in evoluzione: preferibile ampia e poco profonda (più facile aggiungere voci in orizzontale che livelli verticali).

Variante opposta: "Una struttura gerarchica deve essere stretta e molto profonda" → FALSO

Q8. Le convenzioni interne non devono essere rotte per non disorientare l'utente

☒ ~~VERO~~

☐ FALSO

Motivazione: Le convenzioni interne aiutano l'utente a creare uno schema mentale del sito, aumentando familiarità e riducendo carico cognitivo. Romperle causa disorientamento, frustrazione, errori. Esempi: posizione costante navigazione, colori consistenti per azioni, terminologia uniforme, layout ripetibile.

Concetto correlato: Convenzioni **esterne** (standard web) possono essere rotte se c'è beneficio tangibile per l'utente.

Q9. Il layout a schede crea problemi di manutenibilità nel tempo

☒ ~~VERO~~

☐ FALSO

Motivazione: Il layout a schede (tabs) è poco flessibile e difficile da mantenere per siti in evoluzione. Si sviluppa orizzontalmente rischiando: (1) scroll orizzontale indesiderato con molte schede; (2) difficoltà aggiungere nuove sezioni; (3) problemi responsive su mobile; (4) schede "nascoste" se troppe. Adatto solo per: numero fisso schede (max 5-6), contenuti correlati sotto stesso compito.

Variante: "Il layout a schede è facilmente mantenibile e adattabile" → FALSO

Q10. La presenza di una barra breadcrumb evita il sovraccarico cognitivo

- ☐ VERO
☒ FALSO

Motivazione: Il breadcrumb aiuta l'orientamento rispondendo a "Dove sono?" ma **non evita completamente** il sovraccarico cognitivo. L'utente deve comunque: comprendere struttura sito, processare informazioni pagina, gestire navigazione, prendere decisioni. Breadcrumb è uno strumento tra tanti per ridurre (non eliminare) il carico cognitivo.

Q11. È buona regola segnalare link che portano a elementi non web (PDF, file)

- ☒ VERO
☐ FALSO

Motivazione: **Non tradire mai le aspettative dell'utente** (principio fondamentale usabilità). Segnalare esplicitamente: tipo file (PDF, DOC), dimensione (PDF 2.5 MB), link esterni, apertura nuova finestra, cambio contesto. Esempio: Scarica brochure [PDF, 2.5 MB] o icona dedicata. Evita frustrazione e sorprese negative.

● BLOCCO C: Separazione e SEO

Q12. La divisione contenuto/presentazione abbassa il peso totale del sito

- ☒ VERO
☐ FALSO

Motivazione: CSS esterno riutilizzabile su più pagine elimina ridondanza. File CSS cacheable dal browser = scaricato una volta sola. Separazione riduce: byte per pagina, banda necessaria, tempo caricamento. HTML semantico più leggero senza stili inline. Risultato: sito più veloce, meno server load, migliore esperienza mobile.

Q13. La divisione contenuto/presentazione è condizione necessaria e sufficiente per l'accessibilità

- ☐ VERO
☒ FALSO

Motivazione: È necessaria ma non sufficiente. Separazione corretta è base accessibilità ma servono anche: HTML semantico, testi alternativi, contrasto colori, navigabilità tastiera, ARIA quando necessario, struttura logica, ordine DOM corretto, skip links, focus visibile.

Nota: Attenzione a varianti che dicono "contenuto/comportamento" o "struttura/comportamento" invece di "contenuto/presentazione".

Q14. La separazione struttura/presentazione migliora il posizionamento SEO

- ☒ ~~VERO~~
☐ FALSO

Motivazione: I crawler leggono i primi byte della pagina. Con CSS esterno, il contenuto significativo appare subito nell'HTML. Con style inline/embedded, i crawler perdono tempo su CSS prima di trovare contenuto. Inoltre: HTML semantico meglio indicizzabile, velocità caricamento (fattore ranking), mobile-friendly, accessibilità (correlata a SEO).

Q15. La presenza di una parola in meta keywords è sufficiente per modificare il posizionamento SERP

- ☐ VERO
☒ ~~FALSO~~

Motivazione: Meta keywords è **completamente ignorato** da Google dal 2009 (e da altri motori) a causa dell'abuso storico. Non è neanche condizione necessaria. Fattori SEO reali: title tag, h1-h6, contenuto qualità, backlinks, velocità, mobile-friendly, Core Web Vitals, struttura URL, internal linking.

Q16. Il meta tag description influenza il posizionamento nelle SERP

- ☐ VERO
☒ ~~FALSO~~

Motivazione: Description **NON influenza il ranking** (non è fattore posizionamento). Influenza però il **CTR (Click-Through Rate)**: appare come snippet nei risultati, deve convincere l'utente a cliccare. Descrizione efficace: max 145-160 caratteri, include CTA (Call To Action), contiene keyword naturalmente, unica per pagina. Impatto indiretto su SEO tramite comportamento utente.

Q17. Il contenuto del meta tag author influenza il posizionamento SEO

- ☐ VERO
☒ FALSO

Motivazione: Tag author specifica solo l'autore del sito web, non ha alcun impatto sul ranking. Meta tag generalmente irrilevanti per SEO moderno: author, generator, copyright, revised. Meta tag ancora rilevanti: title (cruciale), description (per CTR), viewport (mobile), robots, canonical.

Q18. I titoli delle pagine devono contenere informazioni dal generale al particolare

- ☐ VERO
☒ FALSO

Motivazione: È l'opposto! I title devono andare **dal particolare al generale**: Nome Pagina - Sezione - Nome Sito. Motivi: (1) nei risultati ricerca, se title lungo viene troncato, parte più importante (particolare) resta visibile; (2) nei bookmark, si vede subito contenuto specifico; (3) nelle schede browser con poco spazio, nome pagina appare prima.

Esempio corretto: Ricette Carbonara - Primi Piatti - CucinaItaliana.it

Esempio sbagliato: CucinaItaliana.it - Primi Piatti - Ricette Carbonara



SEZIONE 2: CALCOLO SPECIFICITÀ CSS



Formula Specificità

```
(important, inline, id, attributi/classi/pseudo-classi, tag/pseudo-elementi)
      1           1           n                n                        n
```

Regole priorità:

1. !important (solo sulla proprietà specifica)
 2. Style inline (attributo style="")
 3. Numero #id
 4. Numero .classi, [attributi], :pseudo-classi
 5. Numero tag e ::pseudo-elementi
 6. **A parità:** vince l'ultima regola definita nel CSS
-

ESERCIZIO 1: Specificità base

```
p {  
  color: black;  
  font-size: 14px;  
}  
  
.testo {  
  color: blue;  
  font-size: 16px;  
}  
  
#main p {  
  color: red;  
}  
  
p.testo {  
  font-size: 18px;  
}
```

```
<div id="main">  
  <p class="testo">Questo è il testo</p>  
</div>
```

Domanda: Quale colore e font-size ha il paragrafo?

Soluzione:

1. `p` → (0, 0, 0, 0, 1)
2. `.testo` → (0, 0, 0, 1, 0)
3. `#main p` → (0, 0, 1, 0, 1) ← **VINCE per color**
4. `p.testo` → (0, 0, 0, 1, 1) ← **VINCE per font-size**

Risposta:

- **color:** red (da `#main p`)
- **font-size:** 18px (da `p.testo`)

ESERCIZIO 2: Con !important

```
div.container p {  
  color: green;  
  font-size: 14px;  
}
```

```
#main p {
  color: blue !important;
}

.container p {
  color: red;
  font-size: 16px;
}
```

```
<div id="main" class="container">
  <p>Testo esempio</p>
</div>
```

Soluzione:

1. `div.container p` → (0, 0, 0, 1, 2)
2. `#main p` → (0, 0, 1, 0, 1) + `!important` su `color` → (1, 0, 1, 0, 1)
3. `.container p` → (0, 0, 0, 1, 1)

Risposta:

- **color:** blue (!important batte tutto)
- **font-size:** 14px (da `div.container p` con specificità 0,0,0,1,2)

ESERCIZIO 3: Tabella accessibile (simile esame)

```
th {
  background-color: #bbb;
  color: #000;
}

th[abbr="Lun"] {
  background-color: #555;
  color: #f00;
}

table tr th.lun {
  background-color: #333;
  color: #fff;
}
```

```
<th class="lun" abbr="Lun" scope="col">Lunedì</th>
```


Domanda: Colore testo e sfondo per "Lunedì"?

Soluzione:

1. `th` → (0, 0, 0, 0, 1)
2. `th[abbr="Lun"]` → (0, 0, 0, 1, 1)
3. `table tr th.lun` → (0, 0, 0, 1, 3) ← **VINCE**

Risposta:

- **color:** #fff (bianco)
 - **background-color:** #333 (grigio scuro)
-

ESERCIZIO 4: Mix complesso

```
body div p {  
  color: black;  
}  
  
div > p {  
  color: blue;  
}  
  
.contenuto p {  
  color: green !important;  
}  
  
#articolo p {  
  color: red;  
}
```

```
<div id="articolo" class="contenuto">  
  <p>Paragrafo</p>  
</div>
```

Soluzione:

1. `body div p` → (0, 0, 0, 0, 3)
2. `div > p` → (0, 0, 0, 0, 2)
3. `.contenuto p` → (0, 0, 0, 1, 1) + `!important` → **(1, 0, 0, 1, 1)**
4. `#articolo p` → (0, 0, 1, 0, 1)

Risposta: color: green (!important vince sempre)



ESERCIZIO 5: Pseudo-classi e pseudo-elementi

```
a {
    color: blue;
    text-decoration: none;
}

a:hover {
    color: red;
}

#nav a {
    color: green;
}

#nav a:hover {
    color: orange;
    text-decoration: underline;
}
```

Domanda: Colore link in `<nav id="nav"><a href="#">Link</a></nav>` al passaggio del mouse?

Soluzione (stato :hover):

1. `a:hover` → (0, 0, 0, 1, 1)
2. `#nav a:hover` → (0, 0, 1, 1, 1) ← **VINCE**

Risposta:

- **color: hover:** orange
- **text-decoration: hover:** underline



SEZIONE 3: DOMANDE APERTE TEORIA

Q1. Differenza tra GET e POST (sempre richiesta!)

Risposta strutturata:

GET e POST sono metodi HTTP per inviare dati client→server:

Metodo GET:

- Parametri **visibili nell'URL**: `pagina.php?nome=Mario&eta=25`
- **Limite lunghezza**: ~2048 caratteri (dipende da browser/server)
- Dati in **cronologia browser** e **log server** (tracciabile)

- **Cacheable** dal browser
- **Bookmarkable** (salvabile nei preferiti)
- **Idempotente**: ripetere richiesta non modifica stato server
- **Uso**: recupero dati, ricerche, filtri, paginazione, link condivisibili

Metodo POST:

- Parametri nel **body HTTP** (non visibili nell'URL)
- **Nessun limite** dimensione dati (dipende da configurazione server)
- Dati **NON** in cronologia browser
- **Non cacheable**
- **Non bookmarkable**
- **Non idempotente**: può modificare stato server
- **Uso**: invio form, upload file, login, operazioni CRUD (create/update/delete), dati sensibili

Tabella comparativa:

Caratteristica	GET	POST
Visibilità dati	URL	Body HTTP
Limite dimensione	~2KB	Illimitato
Sicurezza	Bassa	Alta
Cacheable	Sì	No
Bookmarkable	Sì	No
Idempotente	Sì	No
Uso tipico	Lettura	Scrittura

Esempi pratici:

- GET: `www.shop.com/prodotti?categoria=libri&ordine=prezzo`
- POST: Form registrazione, upload foto profilo, aggiornamento password

Q2. Metafora della pesca e tipologie utenti (ricorrente!)

Risposta completa:

La metafora della pesca descrive come gli utenti cercano informazioni, distinguendo **4 comportamenti**:

1. Tiro perfetto (Perfect Catch):

- Utente sa **esattamente** cosa cerca

- Va diretto all'obiettivo
- Esempio: cercare "orari biblioteca centrale Padova"
- **Soluzione design:** motore ricerca efficace, risultati precisi, autocomplete, suggerimenti

2. Trappola per aragoste (Lobster Trap):

- Utente ha **idea abbastanza chiara**
- Vuole apprendere durante la ricerca
- Raffina progressivamente
- Esempio: "smartphone fascia media buona fotocamera"
- **Soluzione design:** filtri progressivi, confronto prodotti, suggerimenti correlati, breadcrumb

3. Pesca con rete (Net Fishing):

- Utente **NON** ha idea precisa
- Esplora, naviga, scopre
- "Non lascio nulla di intentato"
- Esempio: navigare sezione "nuovi arrivi" senza target specifico
- **Soluzione design:** navigazione chiara, categorie intuitive, contenuti evidenziati, serendipity

4. Boa di segnalazione (Marker Buoy):

- Utente vuole **ritrovare** qualcosa già visto
- Bookmarking, cronologia
- Esempio: "dove avevo visto quel corso gratuito?"
- **Soluzione design:** cronologia chiara, preferiti, "visti di recente", URL significativi

Implicazioni progettuali:

- Fornire **ENTRAMBE** le modalità accesso: ricerca (tiro perfetto, aragoste) + navigazione (rete, boa)
- Ricerca visibile in ogni pagina
- Navigazione gerarchica intuitiva
- Breadcrumb per orientamento
- Filtri progressivi per raffinamento

Q3. Profondità massima struttura gerarchica

Risposta:

Profondità ottimale: 4-5 livelli (eccezionalmente 7 per info molto specifiche)

Motivazione:

- Profondità = numero clic da home a pagina target
- Regola empirica: **massimo 3 clic** per info comuni, fino a 5-7 per contenuti specialistici
- Utente frustrato dopo 3-4 clic senza trovare
- Profondità eccessiva: troppi clic, abbandono, disorientamento

Trade-off profondità/ampiezza:

- **Troppo profonda:** molti clic, frustrazione
- **Troppo ampia:** sovraccarico cognitivo, menù con troppe voci (max 7-10)

Struttura ideale: ampia e poco profonda (per siti in evoluzione)

- Facilita aggiunta nuove sezioni in orizzontale
- Riduce modifiche struttura verticale
- Migliore per SEO (page depth)

Esempio pratico:

-  Home → Categoria → Sottocategoria → Prodotto (3 livelli)
 -  Home → Div1 → Div2 → Div3 → Div4 → Div5 → Prodotto (6 livelli)
-

Q4. Numero massimo voci in un menù

Risposta:

Ideale: massimo **7 voci** (eccezionalmente **10**)

Motivazione (Miller's Law):

- Limite cognitivo umano: "7±2" elementi gestibili simultaneamente
- Oltre 7 voci: sovraccarico cognitivo, difficoltà scelta, paralisi decisionale

Problemi con troppi elementi:

1. Aumenta tempo decisione
2. Aumenta probabilità errore
3. Difficoltà visualizzazione mobile
4. Scroll orizzontale indesiderato
5. Utente sopraffatto, abbandona

Soluzioni per molte voci:

- **Categorizzazione:** raggruppare voci correlate

- **Menù dropdown** organizzati gerarchicamente
- **Mega-menù**: per siti grandi (e-commerce)
- **Hamburger menu**: per mobile
- **Ricerca** come alternativa primaria

Nota: Meglio struttura ampia (7-10 voci) ma poco profonda che ristretta e molto profonda (troppi livelli annidati).

Q5. ID vs CLASS in HTML/CSS

Risposta completa:

ID (identificatore):

- **Univoco**: può apparire una sola volta per pagina
- Sintassi CSS: `#nomeID { ... }`
- **Specificità alta**: (0, 1, 0, 0)
- Usato come **ancora**: `<a href="#sezione">Vai a sezione</a>`
- JavaScript: `document.getElementById('header')`
- Deve iniziare con lettera o underscore
- Case-sensitive

CLASS (classe):

- **Riutilizzabile**: applicabile a più elementi
- Sintassi CSS: `.nomeClasse { ... }`
- **Specificità bassa**: (0, 0, 1, 0)
- **Multiplo**: `<div class="card featured large">`
- JavaScript: `document.getElementsByClassName('button')`
- Usata per stilizzazione ricorrente

Tabella differenze:

Aspetto	ID	CLASS
Univocità	Una volta per pagina	Riutilizzabile
Specificità CSS	(0,1,0,0)	(0,0,1,0)
Ancora link	Sì <code>href="#id"</code>	No
JavaScript	<code>getElementById</code>	<code>getElementsByClassName</code>
Multipli su elemento	No	Sì, separati da spazio

Best practices:

- **ID**: elementi strutturali unici (#header , #footer , #main-nav)
- **CLASS**: componenti riutilizzabili (.button , .card , .alert)
- Preferire CLASS per styling (più flessibile)
- Evitare troppi ID (aumenta specificità, difficile override)

Esempio:

```
<!-- Corretto -->
<header id="main-header">
  <button class="btn btn-primary">Login</button>
  <button class="btn btn-secondary">Registrati</button>
</header>

<!-- Sbagliato -->
<div id="button1">Login</div>
<div id="button2" class="button">Registrati</div>
<!-- Altro div con id="button2" → ERRORE validazione -->
```

Q6. Schemi organizzativi: esatti vs ambigui

Risposta:

Gli schemi organizzativi definiscono **come i contenuti sono disposti** nel sito (non la navigazione).

SCHEMI ESATTI (obiettivi, prevedibili):

Utente sa esattamente cosa cerca, organizzazione univoca.

1. Alfabetico: A-Z

- Esempio: elenco contatti, glossario, indice
- Adatto: grandi quantità dati omogenei

2. Cronologico: timeline temporale

- Esempio: archivio news, blog, eventi storici
- Adatto: contenuti con forte componente temporale

3. Geografico: per luogo

- Esempio: uffici per città, itinerari turistici
- Adatto: contenuti legati a località specifiche

SCHEMI AMBIGUI (soggettivi, interpretativi):

Utente non sa esattamente cosa cerca, organizzazione contestuale.

1. **Per argomento/topic:** categorie tematiche
 - Esempio: sito news (Politica, Sport, Economia, Cultura)
 - Adatto: contenuti eterogenei raggruppabili
2. **Orientati al compito/task:** cosa l'utente vuole fare
 - Esempio: sito banca (Apri conto, Richiedi prestito, Paga bollette)
 - Adatto: servizi, applicazioni web, e-commerce
3. **Per audience:** tipologia utente
 - Esempio: sito università (Studenti, Docenti, Personale)
 - Adatto: siti multi-target con esigenze diverse
4. **Metaphor-driven (metaforici):** metafore mondo reale
 - Esempio: "Scrivania", "Carrello", "Cestino"
 - Rischio: metafora non chiara per tutti
5. **Ibridi:** combinazione di più schemi
 - Esempio: news per topic + cronologia
 - Più flessibile ma potenzialmente confusivo

Quale usare?

- **Menù principale:** schema ambiguo (topic o task)
 - **Sotto-sezioni:** possibile schema esatto (alfabetico per elenchi)
 - **Dipende dal contesto** e tipo utente
-

Q7. Tre domande fondamentali per evitare disorientamento

Risposta:

Ogni pagina deve rispondere chiaramente a **3 domande critiche**:

1. DOVE SONO? (orientamento)

- **Title** descrittivo e specifico
- **Logo** sito visibile e cliccabile
- **Breadcrumb** (percorso gerarchico)
- **Evidenziazione** voce menù attiva
- **Intestazione H1** chiara
- **URL** significativo

2. DOVE POSSO ANDARE? (navigazione)

- **Menù principale** sempre visibile/accessibile
- **Link** chiari e descrittivi
- **Pulsanti CTA** evidenti

- **Footer** con link secondari
- **Ricerca** disponibile
- **Numero limitato opzioni** (max 7-10 primarie)

3. DI COSA SI TRATTA? (contenuto)

- **Contenuto principale** above the fold
- **Descrizione chiara** argomento pagina
- **Immagini/media** contestuali
- **Testo introduttivo** esplicativo
- **Meta description** per SERP

Domande aggiuntive (secondarie ma utili): 4. Come sono arrivato qui? → Breadcrumb, referrer 5. Chi gestisce il sito? → About, credenziali 6. Dove trovare approfondimenti? → Link correlati, risorse

Nota: Rispondere correttamente alle prime 3 garantisce assenza di disorientamento utente.

Q8. Cosa mettere "above the fold"? (area sicura visibile senza scroll)

Risposta:

"Above the fold" = area visibile senza scroll al caricamento pagina (circa 600-800px altezza desktop, 500-600px mobile).

Elementi essenziali:

1. Identificazione sito:

- Logo (top-left, cliccabile → home)
- Nome sito/azienda

2. Navigazione principale:

- Menù orizzontale (desktop)
- Hamburger menu (mobile)
- Ricerca (se cruciale)

3. Contenuto informativo principale:

- Headline/titolo H1
- Valore proposta (value proposition)
- Intro contenuto principale
- Immagine hero (se pertinente)

4. Call-to-Action primaria (se applicabile):

- Bottone/link azione principale

- Form contatto (se prioritario)

Principi:

- Massimizza contenuto utile, minimizza header/footer gonfiati
- Mobile first: ancora più critico su piccoli schermi
- Test con device reali (risoluzione varia)
- Non tutto deve stare above fold, ma info critiche sì

Errori comuni:

- Header gigante con immagine decorativa (spreca spazio)
 - Slider carousel inutile (nasconde contenuto)
 - Troppi elementi, nessuno prioritario
-

Q9. Convenzioni interne vs esterne

Risposta:

CONVENZIONI INTERNE (specifiche del sito):

Regole consistenti **all'interno dello stesso sito**.

- **Devono SEMPRE** essere rispettate
- Creano schema mentale utente
- Aumentano familiarità e efficienza

Esempi:

- Posizione costante navigazione
- Colori uniformi per tipologie link
- Terminologia consistente
- Layout pagine simile
- Icone con significato fisso

CONVENZIONI ESTERNE (standard web generali):

Best practices condivise **tra tutti i siti web**.

- **Possono** essere rotte se c'è beneficio tangibile
- Utenti si aspettano comportamenti standard

Esempi:

- Logo top-left → home

- Link blu sottolineati
- Carrello top-right (e-commerce)
- Hamburger menu (mobile)
- Footer con info contatto/privacy

Quando rompere convenzioni esterne?

- **Mai** rompere per capriccio estetico
- Solo se migliora significativamente UX
- Testare con utenti reali
- Valutare trade-off: novità vs confusione

Esempio pratico:

-  Interna: se il tuo sito usa verde per "conferma" in Form A, usalo anche in Form B, C, D
-  Rompere interna: Form A verde = conferma, Form B rosso = conferma → confusione
-  Esterna: rompere "logo top-left" solo se layout innovativo lo giustifica (rischio alto)



SEZIONE 4: TABELLA ACCESSIBILE



Template XHTML STRICT

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="it" lang="it">
<head>
    <title>Menù Settimanale - Mensa Universitaria</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>

<h1>Menù della Settimana</h1>

<table summary="La tabella mostra il menù settimanale della mensa dal lunedì
al venerdì,
        organizzato per portate (primo, secondo, contorno) in 5
colonne giornaliere">
    <caption>Menù settimanale mensa universitaria</caption>

    <thead>
        <tr>
            <th scope="col" abbr="Portata">Portata</th>
            <th scope="col" abbr="Lun">Lunedì</th>
```

```

        <th scope="col" abbr="Mar">Martedì</th>
        <th scope="col" abbr="Mer">Mercoledì</th>
        <th scope="col" abbr="Gio">Giovedì</th>
        <th scope="col" abbr="Ven">Venerdì</th>
    </tr>
</thead>

<tbody>
    <tr>
        <th scope="row">Primo piatto</th>
        <td>Pasta al pomodoro</td>
        <td>Risotto ai funghi</td>
        <td>Pasta alla carbonara</td>
        <td>Lasagne al forno</td>
        <td>Gnocchi al pesto</td>
    </tr>
    <tr>
        <th scope="row">Secondo piatto</th>
        <td>Pollo arrosto</td>
        <td>Pesce al forno</td>
        <td>Cotoletta milanese</td>
        <td><span xml:lang="en">Hamburger</span></td>
        <td><span xml:lang="en">Pizza</span> margherita</td>
    </tr>
    <tr>
        <th scope="row">Contorno</th>
        <td>Insalata mista</td>
        <td>Patate al forno</td>
        <td>Verdure grigliate</td>
        <td>Spinaci</td>
        <td>Carote</td>
    </tr>
</tbody>
</table>

</body>
</html>

```

Elementi chiave XHTML: ☒ `summary` nell'attributo `table` (descrizione max 100 caratteri)

- ☒ `caption` per titolo visibile
- ☒ `thead` , `tbody` per struttura semantica
- ☒ `scope="col"` per intestazioni colonna
- ☒ `scope="row"` per intestazioni riga
- ☒ `abbr` per abbreviazioni screen reader
- ☒ `xml:lang="XX"` per termini stranieri
- ☒ Tag sempre chiusi con `</>`

● Template HTML5

```
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Menù Settimanale - Mensa Universitaria</title>
</head>
<body>

<h1>Menù della Settimana</h1>

<p id="descrizioneMenuSettimana">
  La tabella presenta il menù settimanale della mensa dal lunedì al
  venerdì,
  organizzato in 5 colonne per giorno e 3 righe per portata (primo,
  secondo, contorno).
</p>

<table aria-describedby="descrizioneMenuSettimana">
  <caption>Menù settimanale mensa universitaria</caption>

  <thead>
    <tr>
      <th scope="col">Portata</th>
      <th scope="col" abbr="Lun">Lunedì</th>
      <th scope="col" abbr="Mar">Martedì</th>
      <th scope="col" abbr="Mer">Mercoledì</th>
      <th scope="col" abbr="Gio">Giovedì</th>
      <th scope="col" abbr="Ven">Venerdì</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <th scope="row" data-title="Portata">Primo piatto</th>
      <td data-title="Lunedì">Pasta al pomodoro</td>
      <td data-title="Martedì">Risotto ai funghi</td>
      <td data-title="Mercoledì">Pasta alla carbonara</td>
      <td data-title="Giovedì">Lasagne al forno</td>
      <td data-title="Venerdì">Gnocchi al pesto</td>
    </tr>
    <tr>
      <th scope="row" data-title="Portata">Secondo piatto</th>
      <td data-title="Lunedì">Pollo arrosto</td>
      <td data-title="Martedì">Pesce al forno</td>
      <td data-title="Mercoledì">Cotoletta milanese</td>
      <td data-title="Giovedì"><span lang="en">Hamburger</span></td>
```

```

        <td data-title="Venerdì"><span lang="en">Pizza</span>
margherita</td>
    </tr>
    <tr>
        <th scope="row" data-title="Portata">Contorno</th>
        <td data-title="Lunedì">Insalata mista</td>
        <td data-title="Martedì">Patate al forno</td>
        <td data-title="Mercoledì">Verdure grigliate</td>
        <td data-title="Giovedì">Spinaci</td>
        <td data-title="Venerdì">Carote</td>
    </tr>
</tbody>
</table>

</body>
</html>

```

Elementi chiave HTML5: ☒ NO summary (deprecato)

- ☒ Paragrafo `<p id="descrizione">` esplicativo
- ☒ `aria-describedby="descrizione"` nella table
- ☒ `data-title` su ogni cella (responsive design)
- ☒ `lang="XX"` per termini stranieri (NON `xml:lang`)
- ☒ `<meta charset="UTF-8" />`
- ☒ `thead`, `tbody`, `caption`, `scope`

● Esempio con COLSPAN/ROWSPAN

```

<table aria-describedby="descOccupazioneAule">
    <caption>Occupazione aule Torre Archimede</caption>

    <thead>
        <tr>
            <th></th>
            <th colspan="4" scope="colgroup">Torre Archimede</th>
        </tr>
        <tr>
            <th></th>
            <th scope="col" abbr="1A150">Aula 1A150</th>
            <th scope="col" abbr="1C150">Aula 1C150</th>
            <th scope="col" abbr="1AD100">Aula 1AD100</th>
            <th scope="col" abbr="2BC">Aula 2BC</th>
        </tr>
    </thead>

    <tbody>
        <tr>

```

```

<th rowspan="3" scope="rowgroup">Fascia mattutina</th>
<th scope="row" abbr="8-9">8.00-9.00</th>
<td></td>
<td></td>
<td colspan="2">Tutorato studenti</td>
</tr>
<tr>
<th scope="row" abbr="9-10">9.00-10.00</th>
<td rowspan="2">Esame di Tecnologie <span lang="en">Web</span>
</td>
<td><span lang="en">Human and Machine Vision</span></td>
<td></td>
<td></td>
</tr>
<tr>
<th scope="row" abbr="10-11">10.00-11.00</th>
<td><span lang="en">Human and Machine Vision</span></td>
<td></td>
<td>Esame di <span lang="en">Data Mining</span></td>
</tr>
</tbody>
</table>

```

Note colspan/rowspan: ☒ scope="colgroup" per gruppo di colonne

☒ scope="rowgroup" per gruppo di righe

☒ abbr per intestazioni complesse

☒ Celle vuote con <td></td> (mai omettere)

CHECKLIST PRE-ESAME

Vero/Falso

- ☐ 18 pattern memorizzati con motivazioni concise
- ☐ Attenzione a "necessaria vs sufficiente"
- ☐ Keywords: accessibilità, separazione, gerarchia, SEO, WCAG

Specificità CSS

- ☐ Formula (important, inline, id, class/attr, tag)
- ☐ Ultima definita vince a parità
- ☐ 5 esercizi risolti a tempo

Domande Aperte

- ☐ GET vs POST (tabella comparativa)

- ☐ Metafora pesca (4 tipologie)
- ☐ Profondità gerarchia (4-5, max 7)
- ☐ ID vs CLASS (tabella differenze)
- ☐ Schemi organizzativi (esatti vs ambigui con esempi)
- ☐ 3 domande disorientamento

Tabella Accessibile

- ☐ XHTML: summary, xml:lang, scope, abbr
 - ☐ HTML5: aria-describedby, lang, data-title
 - ☐ colspan/rowspan con colgroup/rowgroup
 - ☐ span lang per termini stranieri
 - ☐ Validazione: tag aperti/chiusi correttamente
-



STRATEGIA TEMPORALE D'ESAME

Tempo totale: 75-90 minuti

Fase 1: Lettura rapida (5 min)

- Leggi tutte le domande
- Identifica domande facili (stella ★)
- Identifica domande difficili (cerchio ⚠)

Fase 2: Vero/Falso (20 min)

- Rispondi certezze immediate
- Segna dubbi, torni dopo
- Motivazioni concise (3-4 righe max)
- **Priorità:** rispondere tutte, meglio 0 che -1

Fase 3: Specificità CSS (10 min)

- Metodico: scrivi tuple per ogni regola
- Verifica calcolo due volte
- Identifica proprietà specifiche richieste

Fase 4: Tabella accessibile (25 min) ⚠ **CRUCIALE**

- Leggi attentamente: XHTML o HTML5?
- Scrivi struttura base prima
- Inserisci contenuti
- Valida sintassi: tag chiusi, attributi corretti

- **Priorità massima:** vale 8-10 punti!

Fase 5: Domande aperte (20 min)

- Struttura risposta prima di scrivere
- Punti chiave, esempi, tabelle se utili
- Completa almeno una risposta perfetta
- Se tempo stringe: bulletpoints chiavi

Fase 6: Revisione (10 min)

- Rileggi V/F dubbi
- Controlla sintassi tabella
- Verifica calcolo specificità
- NON cambiare risposte sicure

PRIORITÀ PUNTEGGIO

1. **Tabella accessibile** (~10 punti) → 30% totale → **MASSIMA PRIORITÀ**
2. **Domande aperte** (~10 punti) → 30% totale
3. **Vero/Falso** (~10 punti) → 30% totale (attenzione -1!)
4. **Specificità CSS** (~4 punti) → 12% totale

Strategia sicurezza:

- Tabella perfetta + 1 domanda aperta bene = 15 punti (quasi sufficienza)
- Aggiungere 4-5 V/F corretti = 23-25 punti (buon voto)
- Specificità corretta = 27-29 punti (ottimo)

SUGGERIMENTI FINALI

Cosa fare

- ✓ Leggi traccia **attentamente** (XHTML vs HTML5)
- ✓ Gestisci tempo (25 min tabella!)
- ✓ Motivazioni V/F concise ma complete
- ✓ Scrivi codice valido (chiudi tag, attributi corretti)
- ✓ Valida sintassi tabella **prima** di passare oltre

Cosa NON fare

- ✗ Non perdere tempo su domanda impossibile
 - ✗ Non rispondere V/F se dubiti (-1 punti!)
 - ✗ Non scrivere codice inventato (deve validare W3C)
 - ✗ Non confondere XHTML/HTML5 (summary vs aria-describedby)
 - ✗ Non lasciare tabella incompleta (priorità assoluta)
-

Tempo medio per domanda:

- V/F: 3-4 min ciascuna
- Specificità: 10 min totale
- Tabella: 25 min
- Aperte: 10 min ciascuna

Buono studio e in bocca al lupo! 🍀